Volume 2 - Frontend Architecture & Performance

Component Design

Question: How would you design reusable components? #1

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #2

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #3

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #4

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #5

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #6

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #7

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #8

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #9

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #10

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #11

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #12

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #13

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #14

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

Question: How would you design reusable components? #15

Answer: Prefer composition over inheritance, keep APIs small, isolate state, document behavior, and test accessibility.

State Management

Question: When should global state be used? #1

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #2

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #3

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #4

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #5

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #6

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #7

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #8

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #9

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #10

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #11

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #12

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #13

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #14

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Question: When should global state be used? #15

Answer: Use global state for shared application data. Keep local UI state inside components when possible.

Performance

Question: How do you optimize frontend performance? #1

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #2

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #3

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #4

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #5

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #6

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #7

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #8

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #9

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #10

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #11

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #12

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #13

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #14

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.

Question: How do you optimize frontend performance? #15

Answer: Use code splitting, lazy loading, memoization, virtualization, caching, and Web Vitals monitoring.